Implementation-Ready Mathematical Companion to Fixed Sparse-Grid Quadrature Filtering and Its Analytical Gradient

Chak Wong

2026-06-06

Abstract

This note defines a fixed sparse-grid quadrature filter (FixedSGQF) for high-dimensional nonlinear state-space models in which exact nonlinear filtering is unavailable but a deterministic Gaussian-surrogate likelihood is still scientifically useful. The method carries one Gaussian state $\mathcal{N}(m_t, P_t)$, evaluates the nonlinear moments needed for Gaussian projection with a fixed sparse-grid cloud in standardized Gaussian coordinates, and defines an analytical gradient of the same deterministic approximate likelihood scalar that the value routine evaluates. The note is implementation-ready at the algorithm level: it states the cloud-construction rule, the one-step value recursion, the branch-conditioned derivative recursion, the same-scalar finite-difference validity rule, and a worked numerical oracle. The main limitation is explicit. FixedSGQF is not exact nonlinear filtering and it does not preserve general non-Gaussian posterior shape; it is a deterministic Gaussian-surrogate lane whose usefulness depends on whether one carried Gaussian and a fixed sparse-grid moment engine are adequate for the intended regime.

Contents

1 Reader Orientation, Scope, and Approval Target

This document is written for three readers at once:

1. implementation engineers who need an auditable mathematical contract,
2. mathematically trained first-year-PhD-level workers who must implement the algorithm correctly from the note, and
3. general scientific reviewers who need to judge whether the proposal is coherent, bounded, and worth approval.

The note therefore serves two roles at once. It is a mathematical derivation of one deterministic approximate filtering lane, and it is an implementation handoff document for that lane. It is not a general survey of nonlinear filtering, and it is not a full software engineering manual.

Background assumed. The reader is assumed to know multivariate Gaussian notation, basic state-space-model notation, Jacobian-style derivative notation, and the use of a Cholesky factor for a positive-definite covariance matrix. The note does not assume prior familiarity with fixed-branch derivative contracts, sparse-grid cloud merging, or same-scalar finite-difference validation; those are defined here.

What this note provides internally. The note defines:

1. the exact filtering target and the narrower approximate object carried by FixedSGQF,
2. the fixed sparse-grid cloud construction in standardized coordinates,
3. the one-step value recursion and the accumulated deterministic approximate log likelihood,
4. the fixed-branch analytical derivative of that same scalar,
5. the same-scalar finite-difference validity rule, and
6. a worked numerical oracle for value and gradient checking.

What this note does not claim. The note does not claim exact nonlinear filtering, exact nonlinear likelihood evaluation, or faithful preservation of arbitrary posterior multimodality, skewness, or hard global interaction structure. It defines one deterministic Gaussian-surrogate lane and makes its approximation boundary explicit.

How to read this note. Sections 2–5 give the problem statement, the exact-versus-approximate object distinction, and the full runtime path. Sections 7 and 8 define the fixed cloud and the value recursion. Sections 10 and 11 define the saved branch and the analytical gradient. Section 12 defines the finite-difference audit, and Section 13 states what evidence supports implementation correctness versus what evidence supports only approximate scientific usefulness.

2 Problem, Exact Target, and Selected Approximation Lane

The scientific problem is high-dimensional nonlinear filtering. In a general state-space model, the exact Bayesian filter propagates a full conditional law through time. Once the transition or observation map is nonlinear, that law is usually no longer representable by a finite list of moments, and in high dimension exact function-valued propagation is rarely a practical computational object.

The question addressed here is therefore narrower than exact filtering: what deterministic approximation lane is transparent about its compromises, mathematically coherent, and still implementable with a reproducible value and analytical gradient?

The lane selected here is *fixed sparse-grid quadrature filtering* (FixedSGQF). The method makes three deliberate approximation decisions:

1. it carries one Gaussian surrogate $\mathcal{N}(m_t, P_t)$ instead of the full filtering density,
2. it evaluates the nonlinear moments needed by that Gaussian projection with a deterministic sparse-grid quadrature rule in standardized Gaussian coordinates, and
3. it freezes the cloud, merge rule, covariance-factor branch, and veto policy so that the reported gradient differentiates the same deterministic scalar that the reported value routine evaluates.

This note chooses FixedSGQF because it defines an auditable deterministic Gaussian-surrogate scalar, an analytical derivative of that same scalar on a fixed branch, and a moment engine less explosive than dense tensor-product Gaussian quadrature in the intended high-dimensional regime [**?** **?**]. It is not a substitute for richer non-Gaussian density-approximation lanes such as the Zhao–Cui squared tensor-train construction [**?**]; it is the deterministic Gaussian-surrogate lane in a broader program.

3 What the Algorithm Computes

This section is the scope statement an implementer should keep in mind while reading every later formula.

Exact target. The scientific target is the exact nonlinear filtering law

$$p_\theta(x_t \mid y_{1:t}). \tag{3.1}$$

That is a full probability law on $\mathbb{R}^{n_x}$.

Approximate carried object. FixedSGQF does *not* store that full law. It stores only one Gaussian surrogate,

$$X_t \mid y_{1:t} \approx \mathcal{N}(m_t, P_t). \tag{3.2}$$

Accumulated deterministic scalar. At each time step the method forms a Gaussian innovation log-likelihood increment,

$$\widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right\}, \tag{3.3}$$

and then accumulates

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta). \tag{3.4}$$

The analytical gradient in this note is the derivative of the same scalar in (3.4), not the derivative of an adaptive grid-selection procedure.

What is discarded. The approximation in (3.2) discards general non-Gaussian posterior shape. Sparse-grid exactness for selected Gaussian moments is not the same as exact posterior representation.

A scalar warning cell. Let

$$X \sim \mathcal{N}(0, P), \qquad Y = X^2 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, R). \tag{3.5}$$

For $y > 0$ and small R, the exact posterior density can have two separated lobes near $\pm\sqrt{y}$. A Gaussian-projection filter can still compute useful moments of X^2, but after the update it stores only one Gaussian. That is not a bug in quadrature. It is the design choice in (3.2).

4 Object Map and Core Terminology

The table below is the object inventory an implementer should keep open while coding.

symbol	meaning	shape
$\xi^{(r)}$	standardized sparse-grid node	n_x
w_r	accumulated signed weight after merge	scalar
m_t, P_t	carried Gaussian after update	n_x, $n_x \times n_x$
m_t^-, P_t^-	predictive Gaussian before update	n_x, $n_x \times n_x$
C_t, C_t^-	Cholesky factors of P_t and P_t^-	$n_x \times n_x$
$\chi_{t-1}^{(r)}$	placed previous-state cloud point	n_x
$a_t^{(r)}$	transition image of $\chi_{t-1}^{(r)}$	n_x
$\chi_t^{(r)}$	placed predictive cloud point	n_x
$z_t^{(r)}$	observation image of $\chi_t^{(r)}$	n_y
$\bar{z}_t$	predicted observation mean	n_y
S_t	innovation covariance	$n_y \times n_y$
$C_{xz,t}$	state-observation cross covariance	$n_x \times n_y$
v_t	innovation residual $y_t - \bar{z}_t$	n_y
K_t	Gaussian-projection gain	$n_x \times n_y$
$\widehat{\ell}_t^{\mathrm{FSGQ}}$	one-step deterministic approximate log likelihood	scalar
$\dot{\square}$	derivative with respect to one parameter component	matching object

Exact filtering law. The full conditional law $p_\theta(x_t \mid y_{1:t})$.

Carried Gaussian surrogate. The pair (m_t, P_t) used in place of the full law.

Standardized sparse-grid cloud. A fixed list $\{(\xi^{(r)}, w_r)\}_{r=1}^M$ in standard normal coordinates. The cloud is fixed before value evaluation.

Physical cloud points. Points obtained by mapping standardized nodes through the current mean and covariance factor, for example $x = m + C\xi$.

Fixed branch. The saved structural choices under which the value and derivative are both evaluated: cloud, merge rule, ordering, factor convention, veto thresholds, and other declared policies.

Same-scalar derivative. The derivative of the deterministic scalar actually evaluated on the saved branch, not the derivative of a changing adaptive algorithm.

Branch veto. A declared failure exit, such as a predictive covariance or innovation covariance failing the positive-definite branch check.

Implementation conventions. This note fixes three conventions report-wide:

1. every covariance factor is the lower-triangular Cholesky factor with positive diagonal, written $C(P) = \mathrm{chol}(P)$, whenever the declared branch is valid;
2. all expressions written with S_t^{-1} are implemented by linear solves against the Cholesky factor of S_t, not by forming an explicit inverse;

3. if floating-point accumulation yields a covariance that is not exactly symmetric, the implementation first symmetrizes by $(P + P^\top)/2$ and then either accepts the declared branch or returns a veto; it does not silently repair the scalar with hidden jitter, clipping, or floor operations.

5 FixedSGQF in One Pass

A junior implementer should be able to say what the method does after one read:

1. Build a fixed sparse-grid cloud in standardized Gaussian coordinates.
2. Start from an initial Gaussian surrogate (m_0, P_0).
3. At each time step, place the previous cloud using the current factor C_{t-1}, pass those points through the nonlinear transition map, and form predictive moments (m_t^-, P_t^-).
4. If the predictive covariance fails the declared branch, stop and report a predictive-branch failure.
5. Place the predictive cloud using C_t^-, pass those points through the nonlinear observation map, and form $\bar{z}_t$, S_t, and $C_{xz,t}$.
6. If the innovation covariance fails the declared branch, stop and report an innovation-branch failure.
7. Form the one-step Gaussian innovation log likelihood $\widehat{\ell}_t^{\mathrm{FSGQ}}$, update the running total $\widehat{\ell}_T^{\mathrm{FSGQ}}$, and update the carried Gaussian (m_t, P_t).
8. If the updated carried covariance fails the declared branch, stop and report a carried-covariance failure.
9. If gradients are requested, propagate the same fixed-branch derivative recursion using the same cloud and the same branch policies.

The approximation ladder is therefore:

$$\begin{aligned} \text{exact target} &\Longrightarrow p_\theta(x_t \mid y_{1:t}), \\ \text{carried surrogate} &\Longrightarrow \mathcal{N}(m_t, P_t), \\ \text{moment engine} &\Longrightarrow \text{fixed sparse-grid quadrature}, \\ \text{reported scalar} &\Longrightarrow \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta), \\ \text{reported derivative} &\Longrightarrow \nabla_\theta \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta; \mathfrak{B}). \end{aligned} \tag{5.1}$$

6 State-Space Model and Gaussian Projection

Use the additive-noise state-space model

$$X_t = f_\theta(X_{t-1}) + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{6.1}$$

$$Y_t = h_\theta(X_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \tag{6.2}$$

with $X_t \in \mathbb{R}^{n_x}$, $Y_t \in \mathbb{R}^{n_y}$, and parameter $\theta \in \mathbb{R}^p$.

The exact Bayesian prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) \, \mathrm{d}x_{t-1}, \tag{6.3}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{\int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) \, \mathrm{d}x_t}. \tag{6.4}$$

FixedSGQF does not close these exact densities; it performs a Gaussian moment projection.

Assume the predictive state is represented by

$$X_t^- \sim \mathcal{N}(m_t^-, P_t^-). \tag{6.5}$$

Let $Z_t = h_\theta(X_t^-)$. The Gaussian projection uses

$$\bar{z}_t = \mathbb{E}[Z_t], \tag{6.6}$$

$$S_t = R_\theta + \mathbb{E}[(Z_t - \bar{z}_t)(Z_t - \bar{z}_t)^\top], \tag{6.7}$$

$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(Z_t - \bar{z}_t)^\top]. \tag{6.8}$$

Given these moments, define

$$v_t = y_t - \bar{z}_t, \qquad K_t = C_{xz,t} S_t^{-1}, \tag{6.9}$$

$$m_t = m_t^- + K_t v_t, \qquad P_t = P_t^- - K_t S_t K_t^\top. \tag{6.10}$$

The sparse-grid machinery answers how to approximate the expectations in (6.6)–(6.8); it does not change the fact that the carried object is (m_t, P_t).

7 Sparse-Grid Construction in Standardized Coordinates

Let $\xi \sim \mathcal{N}(0, I_{n_x})$. The sparse-grid cloud is built once in standardized coordinates and then mapped into physical coordinates by the current Gaussian factor.

For a one-dimensional standard-normal quadrature rule at level ℓ, write

$$I_\ell[\psi] = \sum_{a=1}^{s_\ell} \omega_{\ell,a} \psi(\zeta_{\ell,a}) \approx \int_{\mathbb{R}} \psi(\xi) \phi(\xi) \, \mathrm{d}\xi. \tag{7.1}$$

For a multi-index $\mathbf{i} = (i_1, \ldots, i_b)$, the tensor rule is

$$I_{\mathbf{i}}[F] = (I_{i_1} \otimes \cdots \otimes I_{i_b})[F]. \tag{7.2}$$

Following **?** , Eq. 26–29], define the active band

$$\mathcal{A}_{b,L} = \{\mathbf{i} \in \mathbb{N}^b : L \le |\mathbf{i}| \le L + b - 1\}, \qquad |\mathbf{i}| = \sum_{j=1}^{b} i_j, \tag{7.3}$$

and the Smolyak coefficient

$$c_{\mathbf{i}} = (-1)^{L+b-1-|\mathbf{i}|} \binom{b-1}{L+b-1-|\mathbf{i}|}. \tag{7.4}$$

Then

$$A_{b,L}[F] = \sum_{\mathbf{i} \in \mathcal{A}_{b,L}} c_{\mathbf{i}} I_{\mathbf{i}}[F]. \tag{7.5}$$

Each tensor point contributes a standardized node and signed weight. Duplicate standardized nodes must be merged by accumulating their signed contributions. The final fixed cloud is

$$\mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}}. \tag{7.6}$$

Cloud-construction contract. The fixed cloud is part of the saved scalar. The univariate rule family, merge tolerance, zero-weight threshold, enumeration order, and final sort order all belong to the target. Two implementations that use the same level but different merge or ordering policies have not necessarily evaluated the same scalar.

7.1 Pseudocode: build the fixed sparse-grid cloud

Routine: `build_fixed_sparse_grid_cloud`
Inputs: dimension b, level L, one-dimensional rule family $\{I_\ell\}$, merge tolerance τ_{merge}, zero-weight threshold τ_{zero}.
Outputs: merged standardized cloud $\mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}}$, plus diagnostics.

1. Enumerate all multi-indices $\mathbf{i} \in \mathcal{A}_{b,L}$ in lexicographic order.
2. For each $\mathbf{i}$, enumerate local tensor indices α in lexicographic order.
3. Form the candidate standardized node $\xi^{\mathbf{i},\alpha}$ and signed contribution $c_{\mathbf{i}} w^{\mathbf{i},\alpha}$.
4. Search the current dictionary for an existing node ξ satisfying $\|\xi^{\mathbf{i},\alpha} - \xi\|_\infty \le \tau_{\text{merge}}$.
5. If found, add the signed contribution to the stored weight for that node; otherwise create a new dictionary entry.
6. After all contributions have been processed, remove entries with accumulated absolute weight below τ_{zero}.
7. Sort the surviving nodes lexicographically and store them in that order.
8. Report at least the weight-total diagnostic $\sum_r w_r$, the number of surviving nodes, and the number of negative accumulated weights.

7.2 A toy cloud oracle

Take $b = 2$ and $L = 2$. Use the level-1 and level-2 one-dimensional rules

$$I_1 : (0, 1), \qquad I_2 : \left\{(-\sqrt{3}, 1/6), (0, 2/3), (\sqrt{3}, 1/6)\right\}. \tag{7.7}$$

The active band contains

$$(1, 1), \qquad (1, 2), \qquad (2, 1), \tag{7.8}$$

with coefficients

$$c_{(1,1)} = -1, \qquad c_{(1,2)} = 1, \qquad c_{(2,1)} = 1. \tag{7.9}$$

After duplicate merge, the cloud is

node ξ	accumulated weight	source components
$(0, 0)$	$1/3$	$(1, 1), (1, 2), (2, 1)$
$(-\sqrt{3}, 0)$	$1/6$	$(2, 1)$
$(\sqrt{3}, 0)$	$1/6$	$(2, 1)$
$(0, -\sqrt{3})$	$1/6$	$(1, 2)$
$(0, \sqrt{3})$	$1/6$	$(1, 2)$

The weights sum to one, and any implementation should reproduce this merged cloud exactly up to the declared merge and zero-weight tolerances.

8 Value Path: Filtering and Deterministic Likelihood

Fix a merged standardized cloud

$$\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M}, \qquad \xi^{(r)} \in \mathbb{R}^{n_x}, \quad w_r \in \mathbb{R}. \tag{8.1}$$

The cloud is fixed before likelihood evaluation.

8.1 Prediction stage

Let $P_{t-1} = C_{t-1}C_{t-1}^{\top}$ on the declared Cholesky branch. Place the previous-state cloud,

$$\chi_{t-1}^{(r)} = m_{t-1} + C_{t-1}\xi^{(r)}, \tag{8.2}$$

push those points through the transition,

$$a_t^{(r)} = f_\theta(\chi_{t-1}^{(r)}), \tag{8.3}$$

and compute the predictive moments

$$m_t^- = \sum_{r=1}^{M} w_r a_t^{(r)}, \tag{8.4}$$

$$P_t^- = Q_\theta + \sum_{r=1}^{M} w_r (a_t^{(r)} - m_t^-)(a_t^{(r)} - m_t^-)^{\top}. \tag{8.5}$$

After (8.5), the implementation symmetrizes by $(P_t^- + P_t^{-\top})/2$ and either accepts the Cholesky branch or returns a predictive-branch veto.

8.2 Observation stage

Factor $P_t^- = C_t^-(C_t^-)^{\top}$ on the declared branch and place the predictive cloud,

$$\chi_t^{(r)} = m_t^- + C_t^- \xi^{(r)}. \tag{8.6}$$

Evaluate the observation map,

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}), \tag{8.7}$$

and compute

$$\bar{z}_t = \sum_{r=1}^{M} w_r z_t^{(r)}, \tag{8.8}$$

$$\Delta_t^{(r)} = z_t^{(r)} - \bar{z}_t, \tag{8.9}$$

$$S_t = R_\theta + \sum_{r=1}^{M} w_r \Delta_t^{(r)} \Delta_t^{(r)\top}, \tag{8.10}$$

$$C_{xz,t} = \sum_{r=1}^{M} w_r (\chi_t^{(r)} - m_t^-) \Delta_t^{(r)\top}. \tag{8.11}$$

If the symmetrized innovation covariance fails the declared branch, the value path returns an innovation-branch veto.

If it passes, define

$$v_t = y_t - \bar{z}_t, \tag{8.12}$$

$$\widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right\}, \tag{8.13}$$

$$K_t = C_{xz,t} S_t^{-1}, \tag{8.14}$$

$$m_t = m_t^- + K_t v_t, \tag{8.15}$$

$$P_t = P_t^- - K_t S_t K_t^\top. \tag{8.16}$$

After the update, the implementation again symmetrizes P_t and either accepts the carried branch or returns a carried-covariance veto.

8.3 Pseudocode: one-step value recursion

Routine: `fixed_sgqf_filter_value_step`

Inputs: previous mean/covariance (m_{t-1}, P_{t-1}), current observation y_t, model objects $f_\theta, h_\theta, Q_\theta, R_\theta$, fixed cloud $\mathcal{C}$, branch thresholds, factor convention.

Outputs: one-step increment $\widehat{\ell}_t^{\mathrm{FSGQ}}$, updated pair (m_t, P_t), and either acceptance diagnostics or a failure record.

1. Symmetrize P_{t-1} and factor it as $P_{t-1} = C_{t-1} C_{t-1}^\top$. If factorization fails, return a factor-branch failure.
2. Place previous-state points $\chi_{t-1}^{(r)}$ and compute transition images $a_t^{(r)}$.
3. Compute m_t^- and P_t^-, then symmetrize P_t^-. If the predictive branch fails, return a predictive-covariance failure.
4. Factor P_t^- as $C_t^- C_t^{-\top}$ and place predictive points $\chi_t^{(r)}$.
5. Compute observation images $z_t^{(r)}$, then $\bar{z}_t$, S_t, and $C_{xz,t}$. Symmetrize S_t. If the innovation branch fails, return an innovation-covariance failure.
6. Form v_t, solve against S_t, compute $\widehat{\ell}_t^{\mathrm{FSGQ}}$, and update the running scalar.
7. Compute K_t, m_t, and P_t. Symmetrize P_t. If the carried branch fails, return a carried-covariance failure.
8. Return $\widehat{\ell}_t^{\mathrm{FSGQ}}$, (m_t, P_t), and diagnostics.

9 Worked One-Step Numerical Oracle

This section is a compact end-to-end oracle for both the value path and the gradient path.

Use the one-dimensional model

$$X_0 \sim \mathcal{N}(m_0, P_0), \qquad m_0 = 0.5, \qquad P_0 = 1, \tag{9.1}$$

$$X_1 = f_\beta(X_0) + \eta_1, \qquad f_\beta(x) = \beta x, \qquad \eta_1 \sim \mathcal{N}(0, Q), \tag{9.2}$$

$$Y_1 = h(X_1) + \varepsilon_1, \qquad h(x) = x^2, \qquad \varepsilon_1 \sim \mathcal{N}(0, R), \tag{9.3}$$

with

$$Q = 0.25, \qquad R = 1, \qquad y_1 = 2, \qquad \beta = 1. \tag{9.4}$$

Use the three-point standard-normal cloud

$$\mathcal{C}_{1,2} = \left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{9.5}$$

Then the one-step value objects are

$$m_1^- = 0.5, \qquad P_1^- = 1.25 = \frac{5}{4}, \tag{9.6}$$

$$\bar{z}_1 = 1.5 = \frac{3}{2}, \qquad S_1 = 5.375 = \frac{43}{8}, \tag{9.7}$$

$$C_{xz,1} = 1.25 = \frac{5}{4}, \qquad v_1 = 0.5 = \frac{1}{2}, \tag{9.8}$$

$$K_1 = \frac{10}{43} \approx 0.2325581395, \qquad m_1 = \frac{53}{86} \approx 0.6162790698, \tag{9.9}$$

$$P_1 = \frac{165}{172} \approx 0.9593023256, \qquad \widehat{\ell}_1^{\mathrm{FSGQ}} \approx -1.7830736342. \tag{9.10}$$

An implementation should reproduce these values up to declared floating-point tolerance before attempting larger cases.

10 Saved Scalar and Same-Scalar Contract

The value and derivative paths are meaningful only if they refer to the same saved scalar.

Plain-language rule. Value, gradient, and finite differences must use the same fixed cloud, the same merge and ordering rules, the same factor family, the same symmetrize-then-veto policy, and the same preprocessing and initial-condition policies. The parameter-dependent numerical objects are recomputed on that same branch; the structural route is what must remain fixed.

Formally, a saved branch is the tuple

$$\begin{aligned} \mathfrak{B} = (&y_{1:T},\ \text{observation preprocessing},\ m_0(\theta), P_0(\theta), \dot{m}_0, \dot{P}_0, \\ &\mathcal{C},\ \text{one-dimensional rules},\ \text{merge tolerance},\ \text{zero-weight rule},\ \text{ordering}, \\ &C(P) = \mathrm{chol}(P),\ \text{symmetrize-then-veto rule},\ \epsilon_S, \epsilon_P, \\ &f_\theta, h_\theta, Q_\theta, R_\theta, D_x f_\theta, \partial_i f_\theta, D_x h_\theta, \partial_i h_\theta, \dot{Q}_\theta, \dot{R}_\theta). \end{aligned} \tag{10.1}$$

The fixed scalar is

$$\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta; \mathfrak{B}) = \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{FSGQ}}(\theta; \mathfrak{B}). \tag{10.2}$$

Frozen versus variable objects. The frozen structural objects are the cloud, merge rule, zero-weight rule, ordering, factor family, thresholds, preprocessing policy, and initial-condition policy. The parameter-dependent numerical objects that are recomputed on that fixed branch include $m_t, P_t, C_t, m_t^-, P_t^-, C_t^-, \bar{z}_t, S_t, C_{xz,t}, K_t$, and $\widehat{\ell}_t^{\mathrm{FSGQ}}$.

Operational branch identity for finite differences. A finite-difference row is branch-valid only if both perturbed evaluations reuse the same structural metadata and follow the same realized acceptance path through each likelihood contribution. If one perturbation triggers a different first failure location, a different accepted-prefix pattern, or a different structural policy, the row is branch-invalid and is not interpreted as derivative evidence.

11 Analytical Gradient of the Fixed Scalar

Why this derivative is nontrivial. The one-step log likelihood at time t depends on objects built by earlier steps of the filter. The derivative must therefore propagate through the previous carried Gaussian, the covariance factors used to place points, the nonlinear transition and observation maps, the innovation covariance, and the posterior update. The branch must be fixed first because otherwise the value and derivative would refer to different targets.

Score objects versus propagation objects. For the current one-step score, the essential derivative objects are $\dot{v}_t$ and $\dot{S}_t$. The derivative of the cross covariance $\dot{C}_{xz,t}$ is not needed to close the current score, but it is needed to form $\dot{K}_t$, and hence to propagate $\dot{m}_t$ and $\dot{P}_t$ to the next time step.

Differentiate with respect to one parameter component θ_i. A dot denotes ∂_i.

The derivative dependency chain is

$$\begin{aligned}\theta &\longmapsto (m_{t-1}, P_{t-1}) \longmapsto C_{t-1} \longmapsto \chi_{t-1}^{(r)} \longmapsto a_t^{(r)} \longmapsto (m_t^-, P_t^-) \\ &\longmapsto C_t^- \longmapsto \chi_t^{(r)} \longmapsto z_t^{(r)} \longmapsto (\bar{z}_t, S_t, C_{xz,t}) \longmapsto (v_t, K_t, \widehat{\ell}_t, m_t, P_t).\end{aligned} \tag{11.1}$$

11.1 Cholesky-branch derivative

If $P = LL^\top$ with positive diagonal and $C(P) = L$, define

$$A = L^{-1}\dot{P}L^{-\top}. \tag{11.2}$$

Let $\Phi(A)$ be the lower-triangular operator

$$\Phi(A)_{jk} = \begin{cases} A_{jk}, & j > k, \\ A_{jj}/2, & j = k, \\ 0, & j < k. \end{cases} \tag{11.3}$$

Then

$$\dot{L} = L\Phi(A). \tag{11.4}$$

This derivative matters operationally because every placed cloud point depends on the covariance factor.

11.2 Prediction sensitivities

Differentiate the placed previous-state points,

$$\dot{\chi}_{t-1}^{(r)} = \dot{m}_{t-1} + \dot{C}_{t-1}\xi^{(r)}, \tag{11.5}$$

and the transition images,

$$\dot{a}_t^{(r)} = D_x f_\theta(\chi_{t-1}^{(r)})\dot{\chi}_{t-1}^{(r)} + \partial_i f_\theta(\chi_{t-1}^{(r)}). \tag{11.6}$$

Then

$$\dot{m}_t^- = \sum_{r=1}^{M} w_r \dot{a}_t^{(r)}, \tag{11.7}$$

$$d_t^{(r)} = a_t^{(r)} - m_t^-, \qquad \dot{d}_t^{(r)} = \dot{a}_t^{(r)} - \dot{m}_t^-, \tag{11.8}$$

$$\dot{P}_t^- = \dot{Q}_\theta + \sum_{r=1}^{M} w_r \left\{ \dot{d}_t^{(r)} d_t^{(r)\top} + d_t^{(r)} \dot{d}_t^{(r)\top} \right\}. \tag{11.9}$$

11.3 Observation sensitivities

Differentiate the predictive placed points,

$$\dot{\chi}_t^{(r)} = \dot{m}_t^- + \dot{C}_t^- \xi^{(r)}, \tag{11.10}$$

then the observation images,

$$\dot{z}_t^{(r)} = D_x h_\theta(\chi_t^{(r)}) \dot{\chi}_t^{(r)} + \partial_i h_\theta(\chi_t^{(r)}), \tag{11.11}$$

and then

$$\dot{\bar{z}}_t = \sum_{r=1}^{M} w_r \dot{z}_t^{(r)}, \tag{11.12}$$

$$\dot{v}_t = -\dot{\bar{z}}_t, \tag{11.13}$$

$$\dot{\Delta}_t^{(r)} = \dot{z}_t^{(r)} - \dot{\bar{z}}_t, \tag{11.14}$$

$$\dot{S}_t = \dot{R}_\theta + \sum_{r=1}^{M} w_r \left\{ \dot{\Delta}_t^{(r)} \Delta_t^{(r)\top} + \Delta_t^{(r)} \dot{\Delta}_t^{(r)\top} \right\}, \tag{11.15}$$

$$\dot{C}_{xz,t} = \sum_{r=1}^{M} w_r \left\{ (\dot{\chi}_t^{(r)} - \dot{m}_t^-) \Delta_t^{(r)\top} + (\chi_t^{(r)} - m_t^-) \dot{\Delta}_t^{(r)\top} \right\}. \tag{11.16}$$

11.4 Innovation score

Proposition 1 (Fixed Gaussian innovation score)**.** *Assume S_t is positive definite and the saved branch is differentiable. Let u_t solve*

$$S_t u_t = v_t. \tag{11.17}$$

Then the derivative of (8.13) *is*

$$\dot{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2} \left[\operatorname{tr}(S_t^{-1} \dot{S}_t) + 2 \dot{v}_t^\top u_t - u_t^\top \dot{S}_t u_t \right]. \tag{11.18}$$

Proof. Use $\mathrm{d} \log\det S = \operatorname{tr}(S^{-1}\,\mathrm{d}S)$ and $\mathrm{d}S^{-1} = -S^{-1}(\,\mathrm{d}S)S^{-1}$. With $u = S^{-1}v$,

$$\mathrm{d}(v^\top S^{-1} v) = 2(\,\mathrm{d}v)^\top u - u^\top(\,\mathrm{d}S)u. \tag{11.19}$$

Substitute $\mathrm{d}v = \dot{v}_t$ and $\mathrm{d}S = \dot{S}_t$ into (8.13). □

11.5 Posterior sensitivity propagation

Differentiate the gain,

$$\dot{K}_t = \dot{C}_{xz,t} S_t^{-1} - K_t \dot{S}_t S_t^{-1}, \tag{11.20}$$

then propagate

$$\dot{m}_t = \dot{m}_t^- + \dot{K}_t v_t + K_t \dot{v}_t, \tag{11.21}$$

$$\dot{P}_t = \dot{P}_t^- - \dot{K}_t S_t K_t^\top - K_t \dot{S}_t K_t^\top - K_t S_t \dot{K}_t^\top. \tag{11.22}$$

The structural summary is: $\dot{v}_t$ and $\dot{S}_t$ close the current score, while $\dot{C}_{xz,t}$, $\dot{K}_t$, $\dot{m}_t$, and $\dot{P}_t$ exist because the derivative state must be carried forward in time.

11.6 Pseudocode: one-step gradient recursion

Routine: `fixed_sgqf_filter_gradient_step`

Inputs: all value-step inputs, plus derivative objects $(\dot{m}_{t-1}, \dot{P}_{t-1})$, first-derivative model routines, and the same saved branch.

Outputs: one-step derivative contribution $\dot{\ell}_t^{\mathrm{FSGQ}}$, updated derivative state $(\dot{m}_t, \dot{P}_t)$, and either acceptance diagnostics or the same failure record as the value path.

1. Reuse the same accepted value-path branch through previous factor, predictive factor, and innovation covariance.
2. Compute $\dot{C}_{t-1}$, then $\dot{\chi}_{t-1}^{(r)}$ and $\dot{a}_t^{(r)}$.
3. Compute $\dot{m}_t^-$ and $\dot{P}_t^-$, then $\dot{C}_t^-$.
4. Compute $\dot{\chi}_t^{(r)}$, $\dot{z}_t^{(r)}$, $\dot{\bar{z}}_t$, $\dot{v}_t$, $\dot{S}_t$, and $\dot{C}_{xz,t}$.
5. Solve $S_t u_t = v_t$ on the accepted innovation branch and evaluate the one-step score derivative with (11.18).
6. Compute $\dot{K}_t$, then propagate $\dot{m}_t$ and $\dot{P}_t$.
7. Return the derivative contribution, the updated derivative state, and diagnostics.

12 Finite-Difference Same-Scalar Check

Finite differences test whether the implementation differentiates the same saved scalar that the value path evaluates.

For parameter component i, define the central difference

$$D_i(h) = \frac{\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta + h e_i; \mathfrak{B}) - \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta - h e_i; \mathfrak{B})}{2h}. \tag{12.1}$$

Use a step ladder such as

$$h \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \max\{1, |\theta_i|\}. \tag{12.2}$$

A row is branch-valid only if both perturbed evaluations reuse the same saved structural metadata and match the base evaluation in their realized accepted/failure stage-time pattern. If either side

changes branch or returns a different veto pattern, the row is branch-invalid and is not interpreted as score evidence.

For valid rows, compare the analytical score G_i against $D_i(h)$ by

$$e_i(h) = \frac{|D_i(h) - G_i|}{\max\{1, |D_i(h)|, |G_i|\}}. \tag{12.3}$$

A practical diagnostic passes when at least two consecutive valid rows satisfy $e_i(h) \le 10^{-5}$ and the smaller-step row is no worse than ten times the larger-step row.

12.1 Pseudocode: same-scalar finite-difference audit

Routine: `same_scalar_fd_check`

Inputs: parameter component i, step ladder, base parameter value, saved branch $\mathfrak{B}$, value routine, analytical score G_i.

Outputs: finite-difference table with branch-valid flags and relative errors.

1. For each step size h, evaluate the value routine at $\theta + he_i$ and $\theta - he_i$ using the same saved structural branch.
2. Record whether both perturbations match the base structural metadata and the realized acceptance path.
3. If either side changes branch, mark the row branch-invalid and do not use it as derivative evidence.
4. If the row is branch-valid, compute $D_i(h)$ and the relative error $e_i(h)$.
5. Report whether two consecutive branch-valid rows meet the declared tolerance rule.

13 Validation, Diagnostics, and What Counts as Evidence

FixedSGQF should be validated as an approximate deterministic likelihood lane, not as exact posterior propagation.

Implementation-correctness evidence. The minimum implementation audit should include:

1. cloud-construction checks, including duplicate-node accumulation, weight totals, negative-weight counts, and exact reproduction of the toy cloud in Section 7.2;
2. linear-Gaussian recovery, where the Gaussian projection recursion should match the Kalman filter up to quadrature and floating-point tolerance;
3. the worked one-step numerical oracle in Section 9;
4. same-scalar finite differences on representative parameter points;
5. signed-weight stability diagnostics for the minimum eigenvalues of symmetrized P_t^-, S_t, and P_t, plus any branch veto.

Approximate scientific-usefulness evidence. Separate from implementation correctness, one may also report:

1. accuracy ladders across sparse-grid levels,
2. comparisons against dense quadrature in small dimension,
3. comparisons against linear-Gaussian truth when available,
4. comparisons against particle or tensor-density references in regimes where richer posterior structure matters,
5. runtime, point count, and memory scaling.

These latter diagnostics support judgments about usefulness of the chosen approximation; they do not by themselves certify correct implementation of the saved scalar.

14 Neighboring Methods and Why FixedSGQF Is the Selected Lane

Nearby filters can be compared by the carried object, the moment engine, and whether they admit the deterministic fixed-scalar derivative contract used here.

method	carried object	moment engine	same-scalar gradient lane	main limitation
EKF	one Gaussian	local linearization	yes, but for the linearized moment engine	weak nonlinear moment fidelity
UKF/CKF	one Gaussian	fixed low-order deterministic points	yes	lower-order point design [**?** **?** **?**]
Dense GHQF	one Gaussian	dense tensor-product Gaussian quadrature	yes	point count grows exponentially in dimension
FixedSGQF	one Gaussian	fixed sparse-grid quadrature	yes	still one Gaussian; branch-sensitive signed-weight moments [**?**]
Adaptive sparse grid	one Gaussian	parameter-dependent active grid	only piecewise, unless frozen first	target changes if the live grid changes [**?**]
Particle / TT lanes	richer non-Gaussian object	samples or density representation	different target class	heavier approximation object; different contract [**?**]

The selection argument of this note is local rather than universal. FixedSGQF is selected here because it jointly provides:

1. a declared deterministic approximate likelihood scalar,
2. an analytical derivative of that same scalar on a fixed smooth branch,
3. a moment engine less explosive than dense tensor-product Gaussian quadrature in the intended high-dimensional regime.

This is not a theorem that FixedSGQF is the best nonlinear filter overall. It is the selection statement for the deterministic Gaussian-surrogate lane treated in this note.

15 Conclusion

This note has defined an implementation-ready mathematical companion for fixed sparse-grid quadrature filtering. The document states the exact target and the narrower carried object, constructs the fixed sparse-grid cloud in standardized coordinates, gives the one-step value recursion, fixes the saved-branch same-scalar contract, derives the branch-conditioned analytical gradient, and states the finite-difference rule needed to check that value and derivative refer to the same deterministic scalar.

The note has also preserved the central non-claim. FixedSGQF is not exact nonlinear filtering, and it does not preserve general non-Gaussian posterior shape. It is a deterministic Gaussian-surrogate lane. Its approval claim is therefore bounded: the claim is not universal optimality, but that this lane is coherent, mathematically explicit, auditable by implementation engineers, and sufficiently explicit for mathematically trained junior workers to implement at the algorithm level.

A Implementation-Oriented Notes for a CS-Facing Appendix

This appendix records software-facing detail that is helpful for implementation engineers but is not mathematically indispensable to the main note.

A.1 Suggested record layout

A practical implementation may distinguish at least four record types:

1. a cloud record containing standardized nodes, accumulated weights, negative-weight diagnostics, and deterministic ordering metadata;
2. a one-step value cache containing placed points, transition images, observation images, and moment objects reused by the gradient path;
3. a branch record containing all fixed structural policies used to define the saved scalar;
4. a failure record of the form $(\text{time}, \text{stage}, \text{reason}, \text{diagnostic values})$.

A.2 Deterministic storage and ordering

Because node ordering is part of the saved scalar contract, implementations should store the merged cloud in a deterministic sorted order after duplicate accumulation. Hash-based storage may be used during construction, but the final serialized object should record the deterministic order actually used by value, gradient, and finite-difference routines.

A.3 Suggested routine surfaces

A software-facing implementation may expose routines analogous to:

1. `build_fixed_sparse_grid_cloud`,
2. `fixed_sgqf_value`,
3. `fixed_sgqf_gradient`,
4. `same_scalar_fd_check`.

The exact API is an engineering choice, but these routine boundaries should remain consistent with the mathematical contract of the main note.